```
import cv2
import numpy as np
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter
from torchvision import transforms
import time
```

```
# === Parameters ===
nx = 512                 # Simulation resolution
nm_per_pixel = 1.0       # Physical size per pixel (nm)
NA = 1.35                # Numerical aperture
wavelength = 193.0       # nm (ArF laser)
threshold = 0.25         # Threshold for resist development
num_iter = 200
sharpness = 100
lr=0.0005
sigma = 2.0  # controls blurring, related to NA & wavelength
```

```
# from google.colab import drive
# drive.mount('/content/drive')
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# === Load and process target mask image ===
image_path = '/content/ILT_line_space_100.png'
mask_target_np = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
mask_target=torch.tensor(mask_target_np, dtype=torch.float32, device=device)
H, W = mask_target_np.shape
fov_X = W * nm_per_pixel
fov_Y = H * nm_per_pixel
fov = max(fov_X, fov_Y)
siz = max(H, W)
pad_H = (siz - H) // 2
pad_W = (siz - W) // 2
padded_img = cv2.copyMakeBorder(
    mask_target_np, pad_H, siz - H - pad_H, pad_W, siz - W - pad_W,
    borderType=cv2.BORDER_CONSTANT, value=0
)
resized_img = cv2.resize(padded_img, (nx, nx), interpolation=cv2.INTER_NEAREST)
resized_img_tensor = torch.tensor(resized_img, dtype=torch.float32, device=device)
resized_img_tensor = resized_img_tensor / 255.0
```

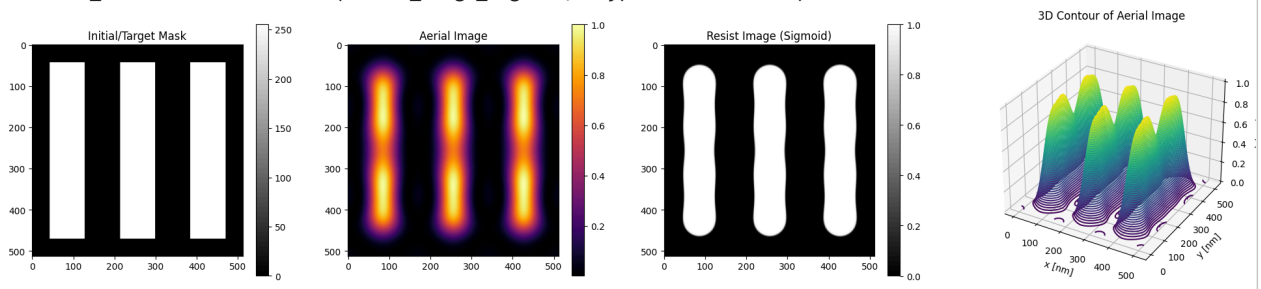
```
# === Forward model: aerial image simulation ===
def compute_aerial_image(weighted_result, NA, wavelength, nm_per_pixel):
    if isinstance(weighted_result, torch.Tensor):
        img = weighted_result.cpu().numpy()
    else:
        img = weighted_result

    fov_x = W * nm_per_pixel
    fov_y = H * nm_per_pixel
    fov = max(fov_x, fov_y)
    size = max(H, W)
    pad_h = (size - h) // 2
    pad_w = (size - w) // 2
    padded_img = cv2.copyMakeBorder(
        img, pad_h, size - h - pad_h, pad_w, size - w - pad_w,
        borderType=cv2.BORDER_CONSTANT, value=0
    )
    resized_img = cv2.resize(padded_img, (nx, nx), interpolation=cv2.INTER_NEAREST)
    mask = torch.tensor(resized_img, dtype=torch.float32)
    mask /= mask.max() if mask.max() > 0 else 1.0

    dx = dy = fov / nx
    fx = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dx))
    fy = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dy))
    FY, FX = torch.meshgrid(fy, fx, indexing='ij')
    mask_ft = torch.fft.fftshift(torch.fft.fft2(mask))
```

```
    cutoff_freq = NA / wavelength
    pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq).float()
    filtered_ft = mask_ft * pupil_filter
    ifft_img = torch.fft.ifft2(torch.fft.ifftshift(filtered_ft))
    aerial_image = torch.abs(ifft_img) ** 2
    aerial_image /= aerial_image.max() if aerial_image.max() > 0 else 1.0

    # Resist development
    aerial_np = aerial_image.cpu().numpy()  # Convert to NumPy
    resist_sigmoid_np = 1 / (1 + np.exp(-sharpness * (aerial_np - threshold)))

    return resist_sigmoid_np, resized_img



def compute_aerial_image_torch(weighted_result, NA, wavelength, nm_per_pixel):
    # Ensure input is a tensor and on the correct device, preserving requires_grad
    if not isinstance(weighted_result, torch.Tensor):
        mask = torch.tensor(weighted_result, dtype=torch.float32, device=device)
    else:
        mask = weighted_result.to(device)

    # Normalize mask if needed, but keep the require_grad property
    if mask.max() > 0:
        mask = mask / mask.max()
    else:
        mask = mask # Avoid division by zero

    # Reuse fov, nx, device from global scope
    dx = dy = fov / nx
    fx = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dx, device=device)) # Ensure on device
    fy = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dy, device=device)) # Ensure on device
    FY, FX = torch.meshgrid(fy, fx, indexing='ij')
    FY = FY.to(device) # Ensure on device
    FX = FX.to(device) # Ensure on device
    # Ensure mask is complex for fft
    mask_complex = mask.to(torch.complex64)

    mask_ft = torch.fft.fftshift(torch.fft.fft2(mask_complex))
    cutoff_freq = NA / wavelength
    pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq).float()
    pupil_filter = pupil_filter.to(device) # Ensure on device
    # with np.printoptions(threshold=np.inf, linewidth=200):
    #     print(pupil_filter.cpu().numpy())
    filtered_ft = mask_ft * pupil_filter
    ifft_img = torch.fft.ifft2(torch.fft.ifftshift(filtered_ft))
    aerial_image = torch.abs(ifft_img) ** 2

    # Normalize aerial image, but maintain require_grad
    if aerial_image.max() > 0:
        aerial_image = aerial_image / aerial_image.max()
    else:
        aerial_image = aerial_image # Avoid division by zero


    # Resist development using PyTorch operations
    resist_sigmoid = 1 / (1 + torch.exp(-sharpness * (aerial_image - threshold)))
    # resist_sigmoid = 0.5 * (1 + torch.cos(aerial_image))
    # resist_sigmoid = torch.clamp(resist_sigmoid, 0.0, 1.0)

    return resist_sigmoid, aerial_image # Return torch tensors
resist_initial_tensor, aerial_initial_tensor= compute_aerial_image_torch(resized_img_tensor, NA, wavelength, nm_p
# # with np.printoptions(threshold=np.inf, linewidth=200):
# #     print(resized_img_512)
# plt.figure(figsize=(6, 6))
# plt.imshow(torch.abs(ifft_img), cmap='gray')
# plt.imshow(aerial_initial_tensor, cmap='gray')
# plt.title('Resized Image (512x512)')
# plt.colorbar()
# plt.show()
```

```
resized_img_tensor_for_aerial = torch.tensor(resized_img, dtype=torch.float32, device=device)

resist_image_sigmoid, resized_target = compute_aerial_image_torch(resized_img_tensor_for_aerial, NA, wavelength,
aerial_tensor = torch.tensor(resist_image_sigmoid, dtype=torch.float32)
# aerial_tensor = torch.tensor(resized_target, dtype=torch.float32)
```

```
x_coords = np.linspace(0, nx * nm_per_pixel, nx)
y_coords = np.linspace(0, nx * nm_per_pixel, nx)
X, Y = np.meshgrid(x_coords, y_coords)


# Visualization
plt.figure(figsize=(24,5))
plt.subplot(1,4,1)
plt.imshow(resized_img, cmap='gray')
plt.title('Initial/Target Mask')
plt.colorbar()

plt.subplot(1,4,2)
plt.imshow(resized_target, cmap='inferno')
plt.title('Aerial Image')
plt.colorbar()

plt.subplot(1,4,3)
plt.imshow(aerial_tensor, cmap='gray')
plt.title('Resist Image (Sigmoid)')
plt.colorbar()

# 4. 3D Contour of Aerial Image
ax = plt.subplot(1, 4, 4, projection='3d')
contour = ax.contour3D(X, Y, resized_target, levels=50, cmap='viridis')
ax.set_title('3D Contour of Aerial Image')
ax.set_xlabel('x [nm]')
ax.set_ylabel('y [nm]')
ax.set_zlabel('Intensity')
plt.show()

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_435/549026905.py:4: UserWarning: To copy construct from a tensor, it is recommended to use sourceTe
  aerial_tensor = torch.tensor(resist_image_sigmoid, dtype=torch.float32)
```

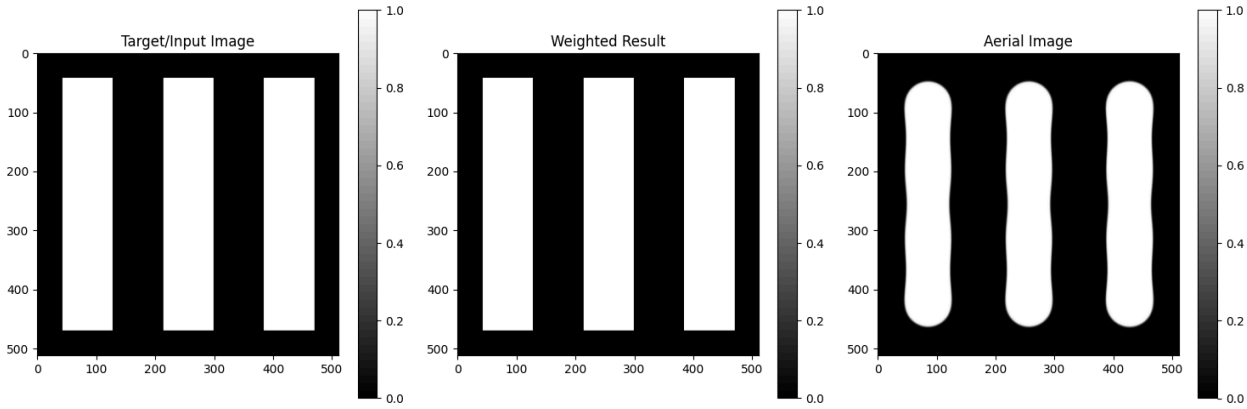



```
<Figure size 640x480 with 0 Axes>
```

```
def angular_weighting(input_tensor: torch.Tensor):
    """Apply angular weighting using theta = arccos(2*input - 1)"""

    # Ensure the input is within the valid domain for arccos [-1, 1]
    # Clamp the input_tensor to the valid range
    clamped_input_tensor = torch.clamp(input_tensor, min=0.0, max=1.0)

    # Calculate theta using specified formula
    theta = torch.acos(2 * clamped_input_tensor - 1) # [-1,1] range ensured

    # Compute angular weighting (1 + cos(theta)) / 2
    angular_weight = (1 + torch.cos(theta)) *0.5

    # Apply weighting
    weighted_image = input_tensor * angular_weight

    return weighted_image, angular_weight
```

```
weighted_result, weight_function = angular_weighting(resized_img_tensor)
res, _ = compute_aerial_image_torch(weighted_result, NA, wavelength, nm_per_pixel)
```

```
# Visualization (convert to numpy for matplotlib)
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
# Ensure mask_np is used here for the original image plot
plt.imshow(resized_img, cmap='gray', vmin=0, vmax=1)
plt.title('Target/Input Image')
plt.colorbar()

plt.subplot(1, 3, 2)
plt.imshow(weighted_result.cpu().numpy(), cmap='gray', vmin=0, vmax=1)
plt.title('Weighted Result')
plt.colorbar()

plt.subplot(1,3,3)
plt.imshow(res.cpu().numpy(), cmap='gray')
plt.title('Aerial Image')
plt.colorbar()

plt.tight_layout()
plt.show()
```




```
def sigmoid(aerial_image, sharpness, threshold):
      # Resist development using PyTorch operations
    resist_sigmoid = 1 / (1 + torch.exp(-sharpness * (aerial_image - threshold)))

    return resist_sigmoid # Return torch tensors
```

USING ARRCOS

```
def optimize_mask(mask_target_tensor: torch.Tensor):
    """
    Optimize the mask such that the aerial image produced by it results in resist output close to that of the tar
    """
    device = mask_target_tensor.device

    # Step 1: Compute initial theta from mask using arccos
    clamped_input = torch.clamp(2 * mask_target_tensor - 1, min=-1.0, max=1.0)
    int_theta = torch.acos(clamped_input)
    int_theta = torch.nan_to_num(int_theta, nan=np.pi)

    # Step 2: Define theta as an optimizable parameter
    theta = torch.nn.Parameter(int_theta.clone().detach().to(device).float())
    optimizer = torch.optim.Adam([theta], lr=0.05)
    loss_history = []

    target = torch.tensor(mask_target_tensor, dtype=torch.float32)
    # Step 3: Compute initial aerial image (optional, for visualization) and target resist image
    with torch.no_grad():
        mask_initial = 0.5 * (1 + torch.cos(theta))
        mask_initial = torch.clamp(mask_initial, 0.0, 1.0)
        target_resist_tensor, aerial_initial_tensor = compute_aerial_image_torch(mask_target_tensor.detach(), NA,
        target_resist = target_resist_tensor
```

```
            aerial_initial = aerial_initial_tensor

        for iter in range(num_iter):
            optimizer.zero_grad()

            # Step 4: Compute current mask from theta
            # current_mask will require grad because theta requires grad
            current_mask = 0.5 * (1 + torch.cos(theta))
            current_weighted_mask = torch.clamp(current_mask, 0.0, 1.0)

            # Step 5: Simulate resist pattern
            # resist_center_tensor will require grad because current_weighted_mask requires grad
            resist_center_tensor, _ = compute_aerial_image_torch(current_weighted_mask, NA, wavelength, nm_per_pixel)
            # Use the tensor directly from the function output
            resist_center = resist_center_tensor

            # Step 6: Loss and backpropagation
            # Compute loss against the pre-calculated target_resist
            # Both resist_center and target_resist are tensors on the device
            loss_mse = F.mse_loss(resist_center, target)
            loss = loss_mse
            # loss now has a grad_fn and can be backpropagated
            loss.backward()
            optimizer.step()

            loss_value = loss.item()
            loss_history.append(loss_value)

            if iter % 10 == 0 or iter == num_iter - 1:
                print(f"[ILT] Iter {iter}: Total={loss_value:.6f}, MSE={loss_mse.item():.6f}")

            # if loss_value < tolerance:
            #     print(f"[ILT] Converged early at iteration {iter} with loss {loss_value:.6f}")
            #     break

        # Final output
        final_mask = 0.5 * (1 + torch.cos(theta.detach()))
            # final_mask = torch.sigmoid(theta.detach())
        final_mask = torch.clamp(final_mask, 0.0, 1.0)
        # resist_final, aerial = compute_aerial_image_torch(final_mask, NA, wavelength, nm_per_pixel)

        # Apply threshold at 0.5
        binary_final_mask = (final_mask > 0.5).float()

        resist_final, aerial = compute_aerial_image_torch(binary_final_mask, NA, wavelength, nm_per_pixel)


    #     return (
    #     final_mask.detach().cpu().numpy(),
    #     loss_history,
    #     aerial_initial.detach().cpu().numpy(), # Keep initial aerial as numpy
    #     resist_final.detach().cpu().numpy(),   # Convert final resist to numpy
    #     aerial.detach().cpu().numpy()         # Convert final aerial to numpy
    # )

        return (
            binary_final_mask.detach().cpu().numpy(),   # binary mask
            loss_history,
            aerial_initial.detach().cpu().numpy(),      # Keep initial aerial as numpy
            resist_final.detach().cpu().numpy(),        # final resist
            aerial.detach().cpu().numpy()              # final aerial
        )
```

```
    # def optimize_mask(mask_target_tensor: torch.Tensor):
    #     """
    #     Optimize the mask such that the aerial image produced by it results in resist output close to that of the t
    #     """
    #     device = mask_target_tensor.device

    #     # Step 1: Parameter initialization. No arccos/cos mapping needed; use sigmoid for smooth [0,1] mapping.
    #     theta = torch.nn.Parameter(mask_target_tensor.clone().detach().to(device).float())
    #     optimizer = torch.optim.Adam([theta], lr=lr)
    #     loss_history = []

    #     target = mask_target_tensor.clone().detach().to(device).float()
```

```
#     # Step 3: Compute initial aerial image (optional, for visualization) and target resist image
#     with torch.no_grad():

#         # mask_initial = torch.sigmoid(aerial_tens)
#         mask_initial = torch.sigmoid(theta)
#         # mask_initial = torch.clamp(mask_initial, 0.0, 1.0)
#         target_resist_tensor, aerial_initial_tensor = compute_aerial_image_torch(mask_target_tensor.detach(), M
#         target_resist = target_resist_tensor
#         aerial_initial = aerial_initial_tensor

#     for iter in range(num_iter):
#         optimizer.zero_grad()

#         # Step 4: Compute current mask from theta using sigmoid for differentiable mapping
#         current_mask = torch.sigmoid(sharpness * (theta - threshold))
#         # current_mask = torch.sigmoid(theta)
#         current_weighted_mask = current_mask  # sigmoid already in [0, 1]

#         # Step 5: Simulate resist pattern
#         resist_center_tensor, _ = compute_aerial_image_torch(current_weighted_mask, NA, wavelength, nm_per_pixe
#         resist_center = resist_center_tensor

#         # Step 6: Loss and backpropagation
#         loss_mse = F.mse_loss(resist_center, target)
#         loss = loss_mse
#         loss.backward()
#         optimizer.step()

#         loss_value = loss.item()
#         loss_history.append(loss_value)

#         if iter % 10 == 0 or iter == num_iter - 1:
#             print(f"[ILT] Iter {iter}: Total={loss_value:.6f}, MSE={loss_mse.item():.6f}")

#     # Final output using sigmoid
#     # final_mask = torch.sigmoid(sharpness * (theta.detach() - threshold))
#         final_mask = torch.sigmoid(theta.detach())
#     resist_final, aerial = compute_aerial_image_torch(final_mask, NA, wavelength, nm_per_pixel)

#     return (
#         final_mask.detach().cpu().numpy(),
#         loss_history,
#         aerial_initial.detach().cpu().numpy(),  # Initial aerial as numpy
#         resist_final.detach().cpu().numpy(),    # Final resist as numpy
#         aerial.detach().cpu().numpy()           # Final aerial as numpy
#     )
```

MAIN

```
# def optimize_mask(mask_target_tensor: torch.Tensor):
#     """
#     Optimize the mask such that the aerial image produced by it results in resist output close to that of the t
#     """
#     device = mask_target_tensor.device

#     # Step 1: Parameter initialization. No arccos/cos mapping needed; use sigmoid for smooth [0,1] mapping.
#     theta = torch.nn.Parameter(mask_target_tensor.clone().detach().to(device).float())
#     optimizer = torch.optim.Adam([theta], lr=lr)
#     loss_history = []

#     target = mask_target_tensor.clone().detach().to(device).float()

#     # Step 3: Compute initial aerial image (optional, for visualization) and target resist image
#     with torch.no_grad():

#         mask_initial = torch.sigmoid(sharpness * (theta - threshold))
#         # mask_initial = torch.sigmoid(theta)
#         mask_initial = torch.clamp(mask_initial, 0.0, 1.0)
#         target_resist_tensor, aerial_initial_tensor = compute_aerial_image_torch(mask_target_tensor.detach(), M
#         target_resist = target_resist_tensor
#         aerial_initial = aerial_initial_tensor

#     for iter in range(num_iter):
```

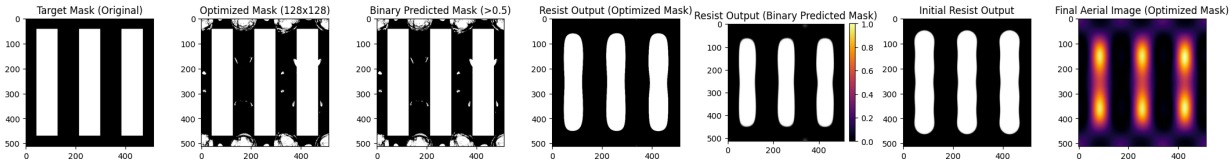
```
#         optimizer.zero_grad()

#         # Step 4: Compute current mask from theta using sigmoid for differentiable mapping
#         current_mask = torch.sigmoid(sharpness * (theta - threshold))
#         # current_mask = torch.sigmoid(theta)
#         current_weighted_mask = current_mask  # sigmoid already in [0, 1]

#         # Step 5: Simulate resist pattern
#         resist_center_tensor, _ = compute_aerial_image_torch(current_weighted_mask, NA, wavelength, nm_per_pixe
#         resist_center = resist_center_tensor

#         # Step 6: Loss and backpropagation
#         loss_mse = F.mse_loss(resist_center, target)
#         loss = loss_mse
#         loss.backward()
#         optimizer.step()

#         loss_value = loss.item()
#         loss_history.append(loss_value)

#         if iter % 10 == 0 or iter == num_iter - 1:
#             print(f"[ILT] Iter {iter}: Total={loss_value:.6f}, MSE={loss_mse.item():.6f}")

#     # Final output using sigmoid
#     # final_mask = torch.sigmoid(sharpness * (theta.detach() - threshold))
#     final_mask = torch.sigmoid(theta.detach())
#     resist_final, aerial = compute_aerial_image_torch(final_mask, NA, wavelength, nm_per_pixel)

#     return (
#         final_mask.detach().cpu().numpy(),
#         loss_history,
#         aerial_initial.detach().cpu().numpy(),  # Initial aerial as numpy
#         resist_final.detach().cpu().numpy(),    # Final resist as numpy
#         aerial.detach().cpu().numpy()           # Final aerial as numpy
#     )
```

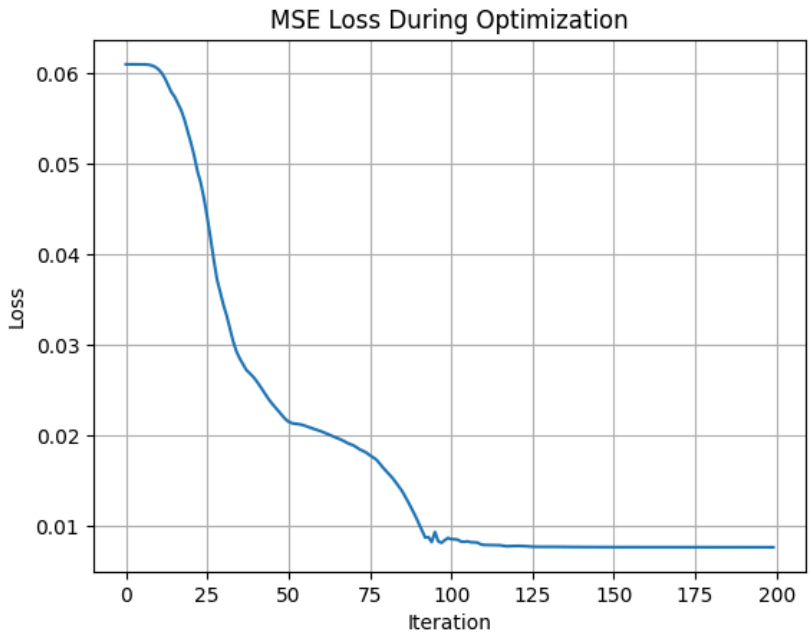
MAIN

```
# Start timer
start_time = time.time()

#Run optimization and get the results
optimized_mask, loss_history, initial_aerial_output, resist_final_output, final_aerial_output = optimize_mask(res

# Apply a threshold of 0.5 to the optimized mask
binary_predicted_mask = (optimized_mask > 0.5).astype(np.float32)

binary_predicted_mask_tensor = torch.tensor(binary_predicted_mask, dtype=torch.float32, device=device)

# Compute the aerial and resist image for the binary predicted mask
# Note: compute_aerial_image_torch expects a tensor
resist_binary_predicted_tensor, aerial_binary_predicted_tensor = compute_aerial_image_torch(binary_predicted_mask

# Convert the resist tensor back to numpy for plotting
resist_binary_predicted_np = resist_binary_predicted_tensor.cpu().numpy()

# Plotting results
fig, ax = plt.subplots(1, 7, figsize=(21, 5)) # Added one subplot for final aerial image

# with np.printoptions(threshold=np.inf, linewidth=200):
#     print(optimized_mask)

# Ensure mask_target_np is used for the original target mask plot
ax[0].imshow(resized_img, cmap='gray')
ax[0].set_title('Target Mask (Original)') # Clarify it's the original

# Use the returned optimized_mask numpy array for plotting
ax[1].imshow(optimized_mask, cmap='gray')
ax[1].set_title('Optimized Mask (128x128)')

# Binary Predicted Mask (Thresholded Optimized Mask)
ax[2].imshow(binary_predicted_mask, cmap='gray')
ax[2].set_title('Binary Predicted Mask (>0.5)')
```

```
with torch.no_grad():
    aerial_np = gaussian_filter(resist_final_output, sigma=sigma)
    # Convert the blurred numpy array back to a torch tensor for sigmoid calculation
    aerial_tens = torch.tensor(aerial_np, dtype=torch.float32, device=device) # Ensure device consistency
    # Compute resist using the blurred aerial image tensor
    resist_tensor = torch.sigmoid(sharpness * (aerial_tens - threshold))
    # resist_tensor = torch.sigmoid(aerial_tens)

ax[3].imshow(resist_tensor, cmap='gray')
ax[3].set_title('Resist Output (Optimized Mask)')

# Plotting the resist image of the binary predicted mask
ax[4].imshow(resist_binary_predicted_np, cmap='gray')
ax[4].set_title('Resist Output (Binary Predicted Mask)')

# Use the passed initial_resist_np for plotting
ax[5].imshow(aerial_tensor, cmap='gray')
# Assuming aerial_image_sigmoid is the initial resist image
ax[5].set_title('Initial Resist Output')

# Add the plot for the final aerial image
im = ax[6].imshow(final_aerial_output, cmap='inferno') # Use the new variable
ax[6].set_title('Final Aerial Image (Optimized Mask)')
fig.colorbar(im, ax=ax[4], fraction=0.046, pad=0.04) # Add colorbar for aerial image

plt.tight_layout()
plt.show()

# Plot loss curve
plt.figure()
plt.plot(loss_history)
plt.title('MSE Loss During Optimization')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.grid(True)
plt.show()

# End timer and print runtime
end_time = time.time()
runtime = end_time - start_time
print(f"Total runtime: {runtime:.2f} seconds")
```

```
/tmp/ipykernel_435/1441051761.py:17: UserWarning: To copy construct from a tensor, it is recommended to use sour
  target = torch.tensor(mask_target_tensor, dtype=torch.float32)
[ILT] Iter 0: Total=0.060945, MSE=0.060945
[ILT] Iter 10: Total=0.060394, MSE=0.060394
[ILT] Iter 20: Total=0.052273, MSE=0.052273
[ILT] Iter 30: Total=0.034401, MSE=0.034401
[ILT] Iter 40: Total=0.026096, MSE=0.026096
[ILT] Iter 50: Total=0.021542, MSE=0.021542
[ILT] Iter 60: Total=0.020479, MSE=0.020479
[ILT] Iter 70: Total=0.018919, MSE=0.018919
[ILT] Iter 80: Total=0.016098, MSE=0.016098
[ILT] Iter 90: Total=0.010365, MSE=0.010365
[ILT] Iter 100: Total=0.008591, MSE=0.008591
[ILT] Iter 110: Total=0.007955, MSE=0.007955
[ILT] Iter 120: Total=0.007847, MSE=0.007847
[ILT] Iter 130: Total=0.007748, MSE=0.007748
[ILT] Iter 140: Total=0.007720, MSE=0.007720
[ILT] Iter 150: Total=0.007707, MSE=0.007707
[ILT] Iter 160: Total=0.007704, MSE=0.007704
[ILT] Iter 170: Total=0.007700, MSE=0.007700
[ILT] Iter 180: Total=0.007697, MSE=0.007697
[ILT] Iter 190: Total=0.007697, MSE=0.007697
[ILT] Iter 199: Total=0.007695, MSE=0.007695
```






```
Total runtime: 9.33 seconds
```

```
abs_difference = np.abs(resist_final_output - aerial_tensor.cpu().numpy())
mae_diff = np.mean(abs_difference)
print(f"MAE between optimized resist and target resist: {mae_diff:.6f}")
```

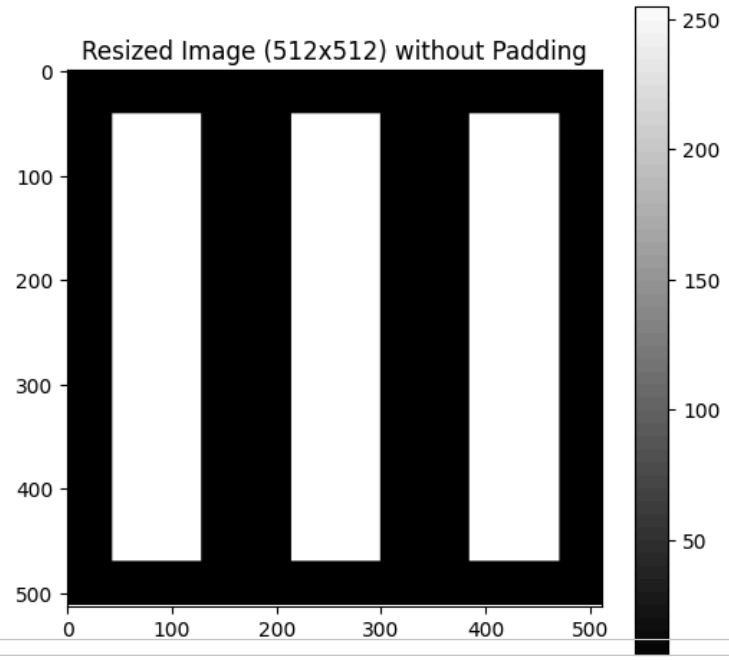
```
MAE between optimized resist and target resist: 0.046629
```

```
def calculate_accuracy_general(mask_np, resist_np, threshold=0.25):
    # Normalize resist if its max value > 1 (assume 0-255 scale)
    if resist_np.max() > 1.0:
        resist_norm = resist_np / 255.0
    else:
        resist_norm = resist_np

    # Binarize resist image for accuracy and precision
    resist_bin = (resist_norm > threshold).astype(np.uint8)
    mask_bin = (mask_np > 0).astype(np.uint8)

    TP = np.sum((mask_bin == 1) & (resist_bin == 1))
    TN = np.sum((mask_bin == 0) & (resist_bin == 0))
```

```
    FP = np.sum((mask_bin == 0) & (resist_bin == 1))

    total_pixels = mask_bin.size

    accuracy = ((TP + TN) / total_pixels) * 100 if total_pixels > 0 else 0
    precision = (TP / (TP + FP)) * 100 if (TP + FP) > 0 else 0

    # Calculate delta on normalized resist and mask (both 0-1)
    # Ensure mask_np is also in the 0-1 range for delta calculation
    mask_norm = mask_np / mask_np.max() if mask_np.max() > 0 else mask_np

    abs_diff = np.abs(resist_norm - mask_norm)
    delta_value = np.mean(abs_diff[mask_bin == 1]) if np.sum(mask_bin == 1) > 0 else 0

    return {
        'accuracy': accuracy,
        'precision': precision,
        'delta_value': delta_value
    }
```

```
# new_image_path = '/content/ILT_line_space_120.png'
# new_mask_np = cv2.imread(new_image_path, cv2.IMREAD_GRAYSCALE)

# if new_mask_np is None:
#     print(f"Error: Could not load image from {new_image_path}")
# else:
#     print(f"Successfully loaded image with shape: {new_mask_np.shape}")

#     # === Padding and Resizing ===
#     H, W = new_mask_np.shape
#     siz = max(H, W)
#     pad_H = (siz - H) // 2
#     pad_W = (siz - W) // 2
#     padded_img = cv2.copyMakeBorder(
#         new_mask_np, pad_H, siz - H - pad_H, pad_W, siz - W - pad_W,
#         borderType=cv2.BORDER_CONSTANT, value=0
#     )
#     resized_img_512 = cv2.resize(padded_img, (512, 512), interpolation=cv2.INTER_NEAREST)

#     print(f"Padded and resized image shape: {resized_img_512.shape}")

#     # Display the resized image
#     plt.figure(figsize=(6, 6))
#     plt.imshow(resized_img_512, cmap='gray')
#     plt.title('Padded and Resized Image (512x512)')
#     plt.colorbar()
#     plt.show()

import cv2
import matplotlib.pyplot as plt

new_image_path = '/content/ILT_line_space_100.png'
new_mask_np = cv2.imread(new_image_path, cv2.IMREAD_GRAYSCALE)

if new_mask_np is None:
    print(f"Error: Could not load image from {new_image_path}")
else:
    print(f"Successfully loaded image with shape: {new_mask_np.shape}")

    # === Resizing to 512x512 without padding ===
    # Note: This will distort the aspect ratio if the original image is not square.
    resized_img_512 = cv2.resize(new_mask_np, (512, 512), interpolation=cv2.INTER_NEAREST)

    print(f"Resized image shape (without padding): {resized_img_512.shape}")

    # Display the resized image
    plt.figure(figsize=(6, 6))
    plt.imshow(resized_img_512, cmap='gray')
    plt.title('Resized Image (512x512) without Padding')
    plt.colorbar()
    plt.show()
```

```
Successfully loaded image with shape: (600, 600)
Resized image shape (without padding): (512, 512)
```




```
metrics_initial = calculate_accuracy_general(resized_img_512, aerial_tensor.cpu().numpy())
metrics_optimized = calculate_accuracy_general(resized_img_512, resist_tensor.cpu().numpy())
metrics_binary_predicted = calculate_accuracy_general(resized_img_512, resist_binary_predicted_np)

print("Initial Contour Metrics:", metrics_initial)
print("Optimized Contour Metrics:", metrics_optimized)
print("Binary Predicted Contour Metrics:", metrics_binary_predicted)
```

```
Initial Contour Metrics: {'accuracy': np.float64(93.94073486328125), 'precision': np.float64(100.0), 'delta_value'
Optimized Contour Metrics: {'accuracy': np.float64(90.35606384277344), 'precision': np.float64(99.94531816506782),
Binary Predicted Contour Metrics: {'accuracy': np.float64(89.45503234863281), 'precision': np.float64(99.943738457
```

```
# Assuming you have already run the cells above to compute metrics_initial, metrics_optimized, and metrics_binary

# Plotting results
fig, ax = plt.subplots(1, 3, figsize=(28, 5))

# Initial Contours
# Remove the batch dimension and convert to numpy for plotting
ax[0].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
ax[0].contour(aerial_tensor, levels=[0.5], colors='red', linewidths=2)
ax[0].set_title(f"Initial Contours: Mask (Blue) vs Resist (Red)\nAccuracy: {metrics_initial['accuracy']:.4f}\nDel
ax[0].axis('off') # Turn off the axis


# Optimized Contours
ax[1].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
ax[1].contour(resist_tensor, levels=[0.5], colors='red', linewidths=2)
ax[1].set_title(f"Optimized Contours: Mask (Blue) vs Resist (Red)\nAccuracy: {metrics_optimized['accuracy']:.4f}\
ax[1].axis('off') # Turn off the axis

# Binary Predicted Contours
ax[2].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
ax[2].contour(resist_binary_predicted_np, levels=[0.5], colors='red', linewidths=2)
```